

UNIVERSIDADE ESTADUAL DE CAMPINAS

IC – Instituto de Computação

MO652 – Computação de Alto Desempenho



Multiplicação de Matrizes em Blocos com MPI e OpenBLAS

João Victor Briganti de Oliveira RA: 215217

Campinas
2026

1 Fundamentação Teórica

Esta seção apresenta a base teórica das implementações de multiplicação de matrizes e suas otimizações, algumas características de precisão em ponto flutuante e a maneira de verificarmos esses erros.

1.1 Multiplicação de matrizes

A multiplicação de matrizes é uma operação de álgebra linear usada em diversas aplicações, como processamento de imagens, processamento de sinais e *machine learning*. Essa operação multiplica uma matriz $m \times k$ por uma matriz $k \times n$ e obtém uma matriz $m \times n$. Cada elemento do resultado é a soma dos produtos de uma linha da primeira matriz pelos elementos de uma coluna da segunda. A Equação (1) mostra um exemplo dessa operação.

$$\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix} = \begin{pmatrix} 1 \cdot 5 + 2 \cdot 7 & 1 \cdot 6 + 2 \cdot 8 \\ 3 \cdot 5 + 4 \cdot 7 & 3 \cdot 6 + 4 \cdot 8 \end{pmatrix} = \begin{pmatrix} 19 & 22 \\ 43 & 50 \end{pmatrix}. \tag{1}$$

Esse algoritmo, apesar de simples, não escala bem conforme a matriz cresce. Seu padrão de acesso não reaproveita os dados que estão na *cache*, o que gera muitos acessos à memória e degrada o desempenho. O *tiling* é uma otimização para esse problema, que divide as matrizes em sub-blocos menores, que reaproveitam melhor a hierarquia de memória do computador [5]. A Equação (2) ilustra essa ideia.

$$\left(\begin{array}{c|c} A_{00} & A_{01} \\ \hline A_{10} & A_{11} \end{array}\right) \times \left(\begin{array}{c|c} B_{00} & B_{01} \\ \hline B_{10} & B_{11} \end{array}\right) = \left(\begin{array}{c|c} A_{00} \cdot B_{00} + A_{01} \cdot B_{10} & A_{00} \cdot B_{01} + A_{01} \cdot B_{11} \\ \hline A_{10} \cdot B_{00} + A_{11} \cdot B_{10} & A_{10} \cdot B_{01} + A_{11} \cdot B_{11} \end{array}\right) \tag{2}$$

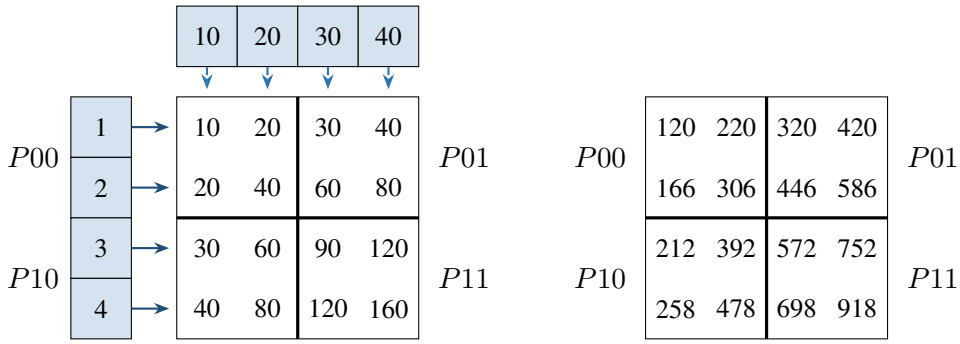
Apesar de ser uma otimização que consegue gerar muitos ganhos o *tiling* depende de parâmetros da arquitetura, como o tamanho do bloco e o da *cache*, e além dele existem outras otimizações que podem gerar ganhos maiores dependendo do problema que você está lidando. Visando padronizar essas rotinas, foi criada a interface *Basic Linear Algebra Subprograms* (BLAS) [1], que permite que cada fabricante forneça uma implementação otimizada das operações de álgebra linear para sua arquitetura, entre elas a multiplicação de matrizes. Neste trabalho usamos o OpenBLAS, que aplica essas otimizações de memória [3] e ainda permite dividir o trabalho entre *threads*.

1.2 Algoritmo SUMMA

O *Message Passing Interface* (MPI) é uma interface de programação para sistemas distribuídos. Nesse modelo, cada processo tem sua própria memória, e a comunicação ocorre por mensagens. Para podermos realizar uma multiplicação de matrizes distribuída, é preciso dividir as matrizes entre os processos e definir a maneira como eles iram se comunicar.

O algoritmo *Scalable Universal Matrix Multiplication Algorithm* (SUMMA) [6] busca resolver esse problema, inicialmente dividindo os processos em uma grade bidimensional, onde cada um guarda um bloco de A, de B e de C. O produto é calculado em passos, e para cada passo (l) ocorrem duas comunicações. Os processos que guardam a coluna l de A, enviam sua parte via *broadcast* aos processos da mesma linha da grade, e os que guardam a linha l de B fazem um processo similar porém em coluna. Com esses dois pedaços, cada processo faz uma multiplicação local e soma o resultado ao seu bloco de C. A Figura 1 mostra o exemplo de uma multiplicação 4×4 em quatro processos. No passo $l = 0$, a coluna $(1, 2, 3, 4)$ de A e a

linha $(10, 20, 30, 40)$ de B chegam a todos, e o processo P_{00}, por exemplo, soma ao seu bloco os produtos $1 \cdot 10$, $1 \cdot 20$, $2 \cdot 10$ e $2 \cdot 20$. Após os quatro passos, cada processo tem seu bloco final de C.



(a) Passo $l = 0$ (b) Resultado C

Figura 1: SUMMA em uma grade 2×2 de processos multiplicando matrizes 4×4. No painel (a), no passo $l = 0$, a coluna 0 de A percorre as linhas da grade (setas contínuas) e a linha 0 de B percorre as colunas (setas tracejadas). Cada célula mostra o produto que o processo soma ao seu bloco de C. No painel (b), após os quatro passos, cada processo guarda um bloco 2×2 de C.

1.3 Ponto flutuante e erro numérico

Um número real pode ter infinitos dígitos, mas o computador tem um número finito de *bits*. Por isso, os números reais precisam ser aproximados, e o padrão *Institute of Electrical and Electronics Engineers* (IEEE) 754 define como essa aproximação e o arredondamento de cada operação são feitos [2]. Um exemplo está na Equação (3), que é um padrão que pode ser observado em qualquer computador que usa esse padrão IEEE, o seu resultado ocorre pois nem $0{,}1$ nem $0{,}2$ são representáveis de forma exata.

$$0{,}1 + 0{,}2 = 0{,}30000000000000004 \tag{3}$$

Cada operação arredonda seu resultado, e a ordem das operações pode acabar agravando esse erro. Somar os mesmos valores em ordens diferentes pode dar resultados diferentes, e é isso que ocorre quando o *tiling*, as *threads* ou os processos dividem uma soma ou multiplicação [4]. Por isso é necessário verificar os resultados paralelos contra uma referência, com uma tolerância para o erro aceito na saída final.

2 Metodologia

Os experimentos foram realizados na fila `paralela` do *cluster* CENAPAD, com oito nós reservados de forma exclusiva, e um total de 1024 núcleos. Cada nó tem dois processadores AMD EPYC 7662 de 64 núcleos, com uma *thread* por núcleo e frequência de aproximadamente 2,0 GHz. Cada nó é dividido em oito regiões *Non-Uniform Memory Access* (NUMA) de 16 núcleos. Os núcleos têm *cache* L1 de dados de 32 KiB e L2 de 512 KiB, a *cache* L3 tem 16 MiB e o conjunto de instruções inclui *Advanced Vector Extensions 2* (AVX2) e *Fused Multiply-Add* (FMA).

O código foi compilado com o GCC 15.2.0 e configurado com o CMake 3.31.8, em modo *Release*. A implementação de MPI utilizada foi o MPICH 5.0.1 [**?**], configurado com o dispositivo CH4 e com a biblioteca *Unified Communication X* (UCX) [**?**] para a comunicação entre os nós. Esses pacotes foram compilados e instalados com o Spack 1.3.0 [**?**]. Para as multiplicações locais foi utilizado o OpenBLAS 0.3.30 com suporte a OpenMP.

Foram avaliadas três versões da multiplicação de matrizes quadradas usando o tipo `float`: a serial, com o algoritmo ingênuo em um único núcleo, a que usa o SUMMA [6] com MPI e aplica a otimização de *tiling*, e a que usa o SUMMA com a rotina `sgemm` do OpenBLAS na multiplicação local. O OpenBLAS aplica otimizações de memória e usa *threads* [3], e é chamado pela interface BLAS [1] em C, com `cblas_sgemm`, na ordem *row-major*, sem transposição. O número de *threads* do OpenBLAS é definido de forma explícita pela variável `OMP_NUM_THREADS`, passada aos processos pela opção `-genv` do `mpirun`.

As matrizes têm tamanho $N \times N$, com $N \in \{1024, 4096, 8192\}$. O número de processos MPI cresce em potências de dois, de 1 até os 1024 núcleos reservados. Na versão MPI sem OpenBLAS, cada processo usa um núcleo e o bloco do *tiling* assume os valores 32, 64 e 128, desde que não seja maior que o bloco local do processo. Na versão com OpenBLAS, cada processo usa também um número de *threads* em potências de dois, de 1 até os 128 núcleos de um nó. O produto entre o número de processos e o de *threads* não ultrapassa o total de núcleos reservados. Cada configuração foi executada três vezes.

O tempo foi medido com a função `clock_gettime` e o relógio `CLOCK_MONOTONIC`, que não sofre ajustes do relógio do sistema, e registrado em nanossegundos. Na versão serial, o trecho cronometrado é apenas a multiplicação de matrizes. Nas versões com MPI, a medição é feita no início da distribuição das matrizes entre os processos (*scatter*) e vai até o fim da coleta do resultado (*gather*), o que inclui a comunicação e o cálculo local.

Ao final do cálculo, é realizado o processo de verificação de corretude que não é medido durante o tempo de análise. Esse processo compara o resultado C com uma referência R, calculada pelo `sgemm` do OpenBLAS. O OpenBLAS foi escolhido como referência por ser mais rápido que a versão serial, o que permitiu verificar mais execuções dentro do tempo da fila. O erro relativo máximo é dado pela Equação (4), e a verificação é bem-sucedida quando ele é no máximo $N \cdot \varepsilon$, em que ε é a diferença entre 1 e o próximo número representável em `float`.

$$\text{erro}_{\text{rel}} = \frac{\max_{i,j} |C_{ij} - R_{ij}|}{\max_{i,j} |R_{ij}|} \leq N \cdot \varepsilon. \tag{4}$$

3 Resultados

Figura 2 mostra o *speedup* das versões MPI e MPI com OpenBLAS em relação à versão serial, em função do número de núcleos. A versão serial levou 97 ms para $N = 1024$, 11,1 s para $N = 4096$ e 95,0 s para $N = 8192$, e esses tempos são a referência de todos os *speedups* desta seção.

A diferença entre os *kernels* já aparece com um único núcleo. Com um processo e uma *thread*, o OpenBLAS é 3,6, 7,7 e 8,2 vezes mais rápido que o laço ingênuo da versão serial, para $N = 1024$, 4096 e 8192. A versão MPI com *tiling*, no mesmo ponto, fica entre 0,6 e 1,2 vezes o desempenho da serial, conforme o tamanho do bloco. O ganho vem da divisão do trabalho, quanto mais processos, menor o tamanho da matriz que eles precisam multiplicar. O melhor *speedup* da versão MPI foi 3,1 (8 processos, bloco 128), 20 (32 processos, bloco 64) e 46 (64 processos, bloco 64). Essa divisão tem um custo, pois a distribuição e a coleta das matrizes e as trocas do SUMMA entram no tempo medido, mesmo com um único processo.

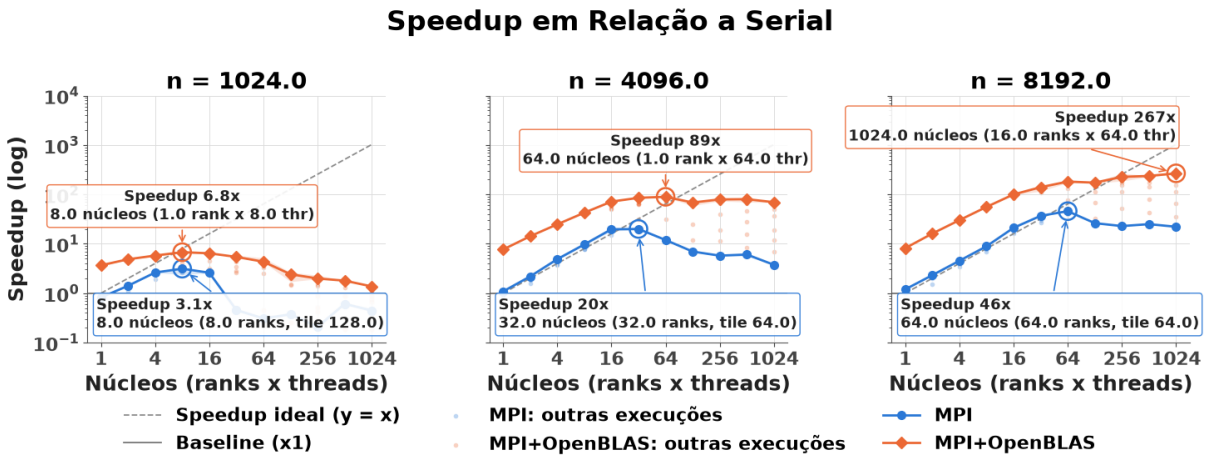


Figura 2: *Speedup* em relação à versão serial, em escala logarítmica. As linhas mostram a média de três execuções e os pontos claros, cada execução. A linha tracejada é o *speedup* ideal.

Os maiores ganhos foram com a versão do OpenBLAS, onde cada processo MPI chama a `sgemm` sobre o seu bloco. Nessa configuração o melhor *speedup* foi 6,8 (1 processo com 8 *threads*), 89 (1 processo com 64 *threads*) e 267 (16 processos com 64 *threads*, 0,36 s). Para $N = 8192$, isso é 5,8 vezes o melhor resultado da versão MPI sem OpenBLAS.

A versão MPI sem OpenBLAS escala até um limite que depende do tamanho da matriz. Para $N = 8192$, o *speedup* chega a 46 com 64 núcleos e cai para 22 com 1024. Para $N = 4096$, o máximo é 20 com 32 núcleos, e com 1024 o valor é 3,7. Para $N = 1024$, o máximo é 3,1 com 8 núcleos, e a partir de 32 núcleos a versão fica mais lenta que a serial. A Figura 3 mostra a mesma tendência na eficiência em relação à própria versão com um núcleo: com 32 núcleos ela é de 96% para $N = 8192$, de 56% para $N = 4096$ e de 2% para $N = 1024$. Esse comportamento é explicado pela maneira como o trabalho de cada processo cresce com a matriz, enquanto a comunicação do SUMMA não escala da mesma forma. Quanto menor a matriz, menos cálculo cada processo tem para compensar a comunicação, o que acaba gerando um *overhead* no tempo total da execução.

A versão com OpenBLAS se sustenta por mais tempo, mas também tem um limite. Para $N = 8192$, o *speedup* cresce até os 1024 núcleos e chega a 267, embora a eficiência em relação a um núcleo caia de 76% com 16 núcleos para 3% com 1024. Para $N = 4096$, o máximo é 89 com 64 núcleos e depois o valor fica entre 66 e 79. Para $N = 1024$, o máximo é 6,8 com 8 núcleos, e com 1024 o *speedup* é 1,4. O limite também depende do tamanho da matriz, mas a eficiência cai mais cedo que na versão MPI, pois o uso de *threads* na computação torna o processo ainda mais rápido o que torna o tempo de cálculo ainda menor, e faz com que a comunicação se torne um gargalo. Por isso, os melhores resultados usam poucos processos e muitas *threads*, como 1 processo com 64 *threads* para $N = 4096$.

Perceba que no gráfico de eficiência vão haver casos onde a versão com MPI escala mais que a serial, e a mesma coisa ocorre com a versão do OpenBLAS. Isso ocorre porque ambos os modelos já possuem uma versão mais rápida que a serial de multiplicação de matrizes, por isso a eficiência em relação a serial passa de 100% com poucos núcleos. O OpenBLAS, mostra essa características de uma maneira mais fácil de visualizar, a `sgemm` é 3,6 a 8,2 vezes mais rápida que a serial com um núcleo, e a eficiência em relação à serial se mantém acima de 100% até 64 núcleos para $N = 4096$ (140%) e até 128 para $N = 8192$ (136%). Mesmo com 1024 núcleos e 3% de eficiência em relação a si mesma, a versão ainda tem 26% em relação à serial para $N = 8192$.

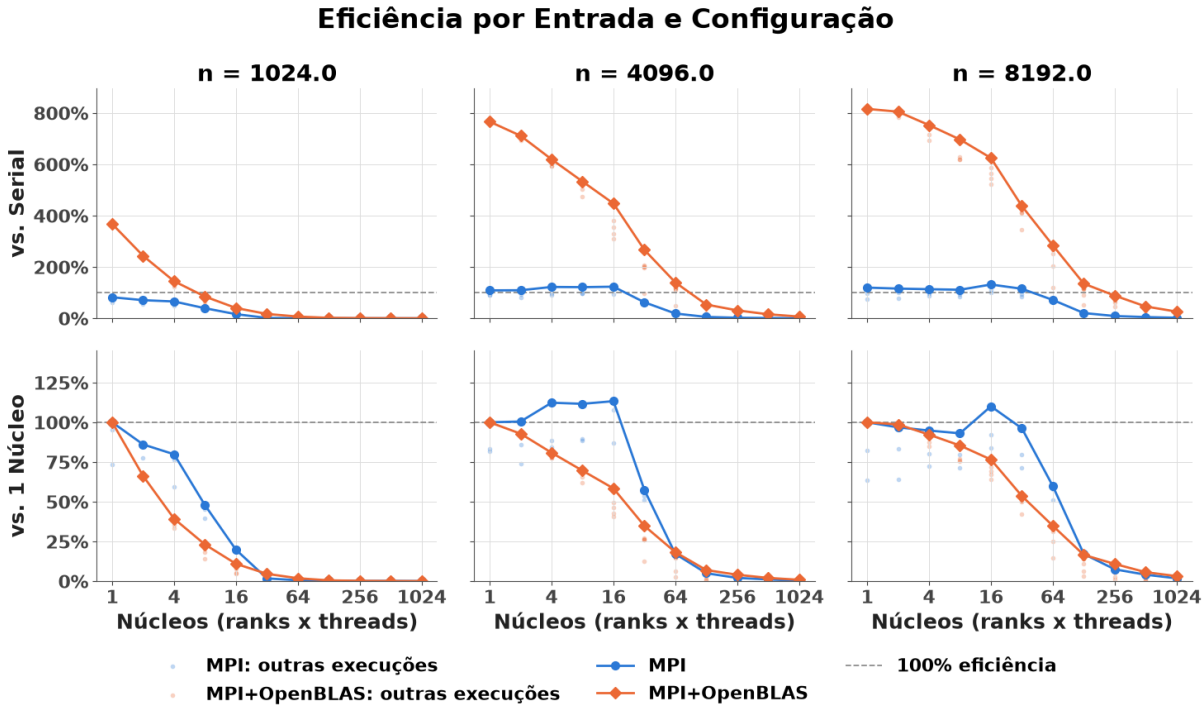


Figura 3: Eficiência em relação à versão serial (acima) e à própria versão com um núcleo (abaixo).

Figura 4 mostra o erro relativo máximo de cada versão, definido na Equação (4). Todas as execuções passaram na verificação, com erros muito abaixo da tolerância $N \cdot \varepsilon$, que vai de $1{,}2 \times 10^{-4}$ para $N = 1024$ a $9{,}8 \times 10^{-4}$ para $N = 8192$. As versões serial e MPI têm o mesmo erro: $2{,}3 \times 10^{-6}$, $4{,}7 \times 10^{-6}$ e $7{,}3 \times 10^{-6}$ para $N = 1024$, 4096 e 8192. Esse valor não depende do número de processos nem do tamanho do bloco. A versão com OpenBLAS tem erros entre $5{,}4 \times 10^{-7}$ e $9{,}2 \times 10^{-7}$, cerca de 12 vezes menores que os da serial para $N = 8192$ ($6{,}2 \times 10^{-7}$ contra $7{,}3 \times 10^{-6}$). Com um único processo, o erro é exatamente zero.

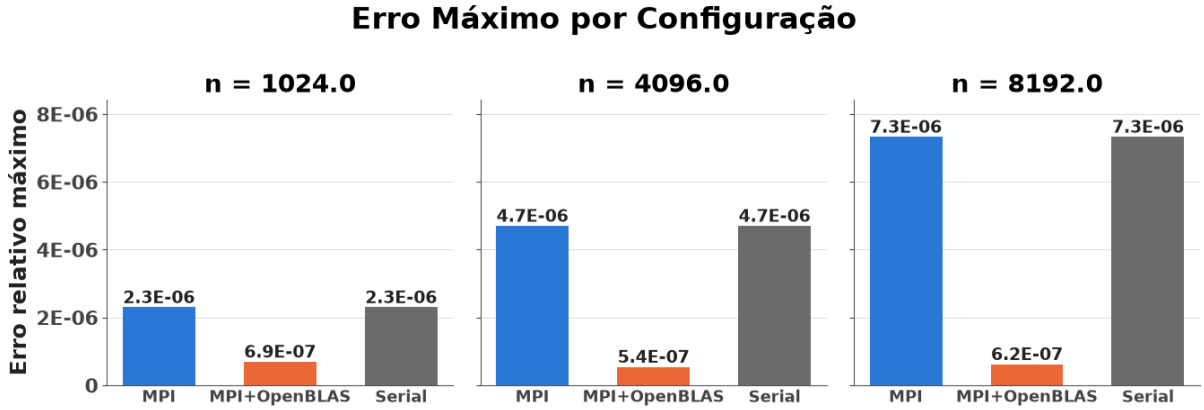


Figura 4: Erro relativo máximo de cada versão em relação à referência calculada pela `sgemm` do OpenBLAS.

A referência é a `sgemm` do OpenBLAS. Ela foi escolhida por ser mais rápida que a versão serial, o que permitiu verificar mais execuções na fila sem exceder o tempo disponível. Como a `sgemm` também usa `float`, ela tem o seu próprio erro de arredondamento, e a diferença só é zero

quando o programa executa a mesma sequência de operações. A versão serial e a MPI somam os produtos de cada elemento em uma ordem diferente da `sgemm`, que usa blocos e instruções vetoriais. Como cada elemento soma N produtos, o arredondamento acumulado cresce com N, o que explica o aumento do erro. A versão MPI soma na mesma ordem que a serial, por isso tem o mesmo erro. Já na versão com OpenBLAS os erros são pequenos porque a soma dos blocos locais tem a mesma estrutura da referência, e o que difere é apenas a combinação dos blocos entre processos.

4 Conclusão

Este trabalho avaliou três versões da multiplicação de matrizes quadradas em `float` no *cluster* CENAPAD, com até 1024 núcleos e matrizes de $N = 1024$, 4096 e 8192. As versões foram a serial, a MPI com SUMMA e *tiling*, e a MPI com SUMMA e `sgemm` do OpenBLAS. O *speedup* foi medido em relação à versão serial. O melhor resultado foi obtido pela versão MPI com OpenBLAS, com *speedup* de 267 para $N = 8192$ (16 processos com 64 *threads*, 0,36 s), contra 46 da melhor versão MPI sem OpenBLAS (64 processos e bloco 64). O principal motivo é o *kernel*, pois com um núcleo a `sgemm` já é de 3,6 a 8,2 vezes mais rápida que o algoritmo ingênuo, e esse ganho se soma à divisão do trabalho entre os processos. A principal limitação dessa versão é que, como o *kernel* é muito rápido, sobra menos cálculo para compensar a comunicação do MPI, e por isso os melhores resultados usaram poucos processos com muitas *threads*. A versão MPI sem OpenBLAS parou de melhorar com 32 a 64 processos e, para $N = 1024$, ficou mais lenta que a serial a partir de 32 núcleos.

Todas as execuções passaram na verificação de corretude, com erros muito abaixo da tolerância. As versões serial e MPI têm o mesmo erro, de $2{,}3 \times 10^{-6}$ ($N = 1024$) a $7{,}3 \times 10^{-6}$ ($N = 8192$), que cresce com N e não depende do número de processos nem do bloco. A versão com OpenBLAS tem erros entre $5{,}4 \times 10^{-7}$ e $9{,}2 \times 10^{-7}$, cerca de 12 vezes menores que os da serial para $N = 8192$, o que torna essa a melhor versão para as cargas de trabalho testadas, pois foi a mais rápida e também a mais precisa.

Referências

[1] J. J. Dongarra, Jeremy Du Croz, Sven Hammarling, and I. S. Duff. A set of level 3 basic linear algebra subprograms. *ACM Transactions on Mathematical Software*, 16(1):1–17, March 1990. ISSN 1557-7295. doi: 10.1145/77626.79170.

[2] David Goldberg. What every computer scientist should know about floating-point arithmetic. *ACM Computing Surveys*, 23(1):5–48, March 1991. ISSN 1557-7341. doi: 10.1145/103162.103163.

[3] Kazushige Goto and Robert A. van de Geijn. Anatomy of high-performance matrix multiplication. *ACM Transactions on Mathematical Software*, 34(3):1–25, May 2008. ISSN 1557-7295. doi: 10.1145/1356052.1356053.

[4] Nicholas J. Higham. *Accuracy and Stability of Numerical Algorithms: Second Edition*. Society for Industrial and Applied Mathematics, January 2002. ISBN 9780898718027. doi: 10.1137/1.9780898718027.

[5] Monica D. Lam, Edward E. Rothberg, and Michael E. Wolf. The cache performance and optimizations of blocked algorithms. In *Proceedings of the fourth international conference*

on Architectural support for programming languages and operating systems - ASPLOS-IV, ASPLOS-IV, page 63–74. ACM Press, 1991. doi: 10.1145/106972.106981.

[6] R. A. Van De Geijn and J. Watts. SUMMA: scalable universal matrix multiplication algorithm. *Concurrency: Practice and Experience*, 9(4):255–274, April 1997. ISSN 1096-9128. doi: 10.1002/(sici)1096-9128(199704)9:4<255::aid-cpe250>3.0.co;2-2.